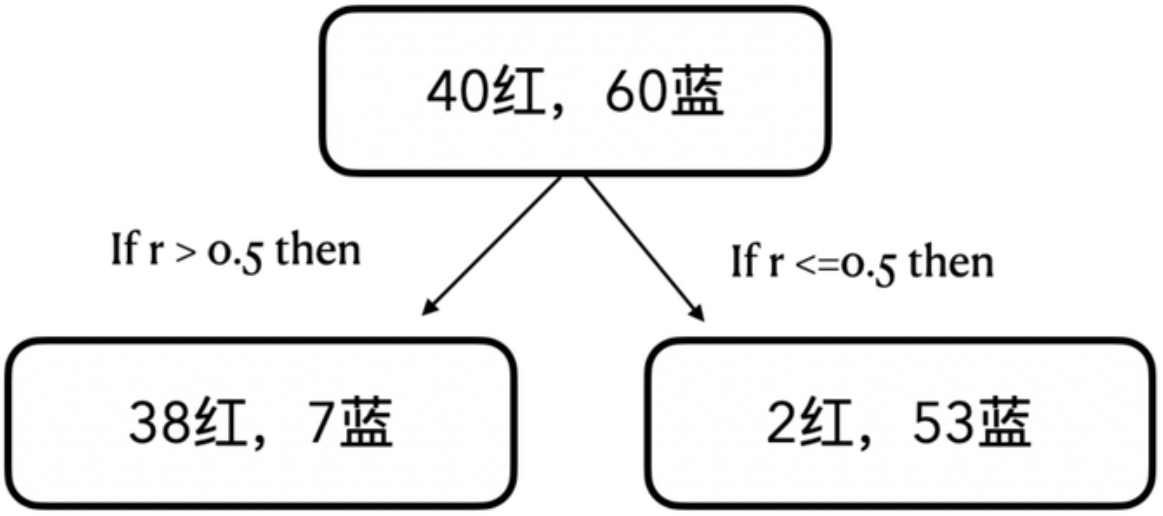
1 - Basic Decision Tree Model

1. Information Entropy

决策树模型是一种 $if-else$ 型的模型，通过对数据集的学习，生成树模型的内部节点，内部节点表示对一个属性的判断。

以一个小球分类问题为例，假设有 100 个小球，其中 40 个为红色球，60 个为蓝色球。而小球有三种特征，分别是表面粗糙度、半径和质量。现在需要依据这三个特征来推算出小球颜色。假设我们尝试使用小球半径这一特征来区分小球，如使用 0.5 作为分界。将这 100 个小球分到两个箱子里，大于 0.5 的在一个箱子里，小于等于 0.5 的在另一个箱子里。我们得到了如下图所示的结果：



凭感觉可以看出，使用这一特征对小球进行区分，看似红和蓝分的更开了。但这不够严谨，而且无法使用数字去比较使用不同特征去对小球分类获得的结果的好坏，也难以比较 r 使用 0.4 或者 0.6 分类，哪个效果更好。为了度量分类结果的好坏，引入了**信息熵**的概念。

从定义上讲，**信息熵用来衡量不确定性，不确定性越强，则信息熵越大，不确定性越弱，则信息熵越小**。其公式如下所示：

$$H(X)=-\sum_{i=1}^{n}P(X=i)log_2P(X=i)$$

以抛硬币为例，可以看出结果越确定，信息熵越小，反之则越大。

硬币	P
正面	0.5
反面	0.5

$$\text{Entropy} = -0.5 * log0.5 - 0.5 * log0.5 = 1$$

硬币	P
正面	0.99
反面	0.01

$$\text{Entropy} = -0.99 * log0.99 - 0.01 * log0.01 = 0.08$$

有了信息熵，我们就可以量化计算之前的小球分类问题了。但为了表述准确，还需要引入两个概念，**条件熵**和**信息增益**。

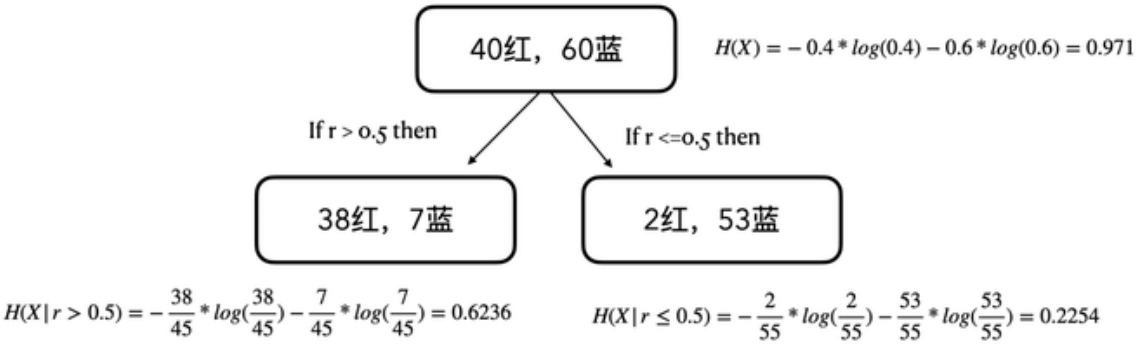
条件熵，定义是在给定随机变量 Y 的条件下，随机变量 X 的不确定性。用通俗的话讲就是，施加一个条件后（如 $r > 0.5$），在这一条件下的信息熵。可以类比条件概率去理解。

$$H(X|Y=v) = -\sum_{i=1}^{n} P(X=i|Y=v) log_2 P(X=i|Y=v)$$

信息增益，是用于衡量给定条件后，信息的不确定性减少了多少。即加了个条件后，信息熵的变化。

$$I(X,Y) = H(X) - H(X|Y)$$

使用这三个概念，再来看小球分类问题。



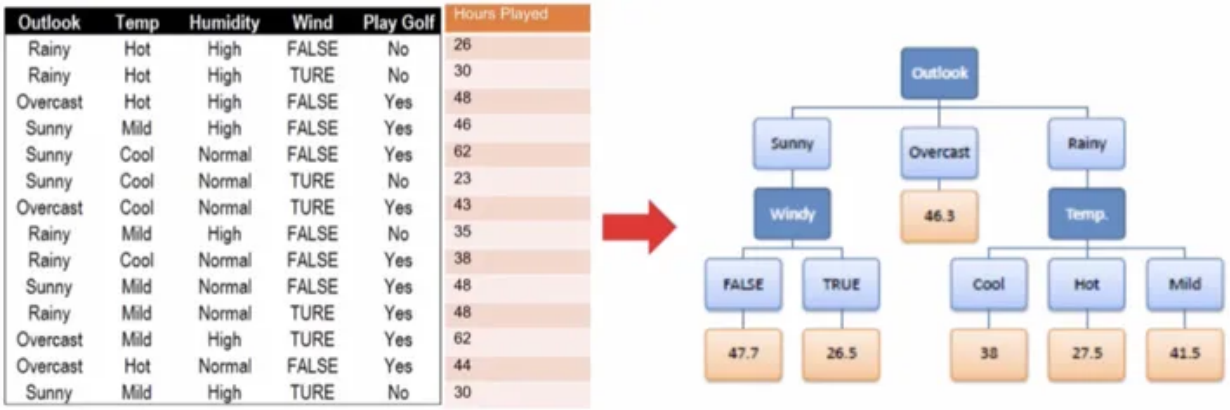
代入公式就可以算出**叶子结点的条件熵**以及**根结点的信息熵**。而**信息增益则是父节点的熵减去子节点的加权熵**。使用信息增益则可以用来衡量这一节点构建的好坏。

$$InformationGain = Father - childs = 0.971 - (0.6236 \times 0.45 + 0.2254 \times 0.55) = 0.5664$$

这样决策树就可以使用信息增益来作为判断依据，使用优化算法通过数据集中的各项特征去建立一个 “最优” 树。

决策树可分为**分类树**和**回归树**，这二者的区别就是叶子结点是**类别**还是**具体的数值**。以下图为例，是一个天气和打高尔夫球之间关系的数据集。如果 $label$ 为是否打高尔夫球，则训练出的是一颗分类树，叶子结点是右图的蓝色节点。如果以打高尔夫球的时间作为

$label$，训练出的就是一颗回归树，叶子结点为黄色部分，模型的输出是打高尔夫的时间。



Outlook	Temp	Humidity	Wind	Play Golf	Hours Played
Rainy	Hot	High	FALSE	No	26
Rainy	Hot	High	TURE	No	30
Overcast	Hot	High	FALSE	Yes	48
Sunny	Mild	High	FALSE	Yes	46
Sunny	Cool	Normal	FALSE	Yes	62
Sunny	Cool	Normal	TURE	No	23
Overcast	Cool	Normal	TURE	Yes	43
Rainy	Mild	High	FALSE	No	35
Rainy	Cool	Normal	FALSE	Yes	38
Sunny	Mild	Normal	FALSE	Yes	48
Rainy	Mild	Normal	TURE	Yes	48
Overcast	Mild	High	TURE	Yes	62
Overcast	Hot	Normal	FALSE	Yes	44
Sunny	Mild	High	TURE	No	30

2. ID3

理解了信息增益和决策树的基本概念之后，就可以开始学习决策树的构建算法。这一部分将借鉴于西瓜书里的例子。以选西瓜为例（数据集的情况见下图），根结点（即原始数据集）中，有 8 个好瓜和 9 个坏瓜，信息熵为 0.998。

编号	色泽	根蒂	敲声	纹理	脐部	触感	好瓜
1	青绿	蜷缩	浊响	清晰	凹陷	硬滑	是
2	乌黑	蜷缩	沉闷	清晰	凹陷	硬滑	是
3	乌黑	蜷缩	浊响	清晰	凹陷	硬滑	是
4	青绿	蜷缩	沉闷	清晰	凹陷	硬滑	是
5	浅白	蜷缩	浊响	清晰	凹陷	硬滑	是
6	青绿	稍蜷	浊响	清晰	稍凹	软粘	是
7	乌黑	稍蜷	浊响	稍糊	稍凹	软粘	是
8	乌黑	稍蜷	浊响	清晰	稍凹	硬滑	是
9	乌黑	稍蜷	沉闷	稍糊	稍凹	硬滑	否
10	青绿	硬挺	清脆	清晰	平坦	软粘	否
11	浅白	硬挺	清脆	模糊	平坦	硬滑	否
12	浅白	蜷缩	浊响	模糊	平坦	软粘	否
13	青绿	稍蜷	浊响	稍糊	凹陷	硬滑	否
14	浅白	稍蜷	沉闷	稍糊	凹陷	硬滑	否
15	乌黑	稍蜷	浊响	清晰	稍凹	软粘	否
16	浅白	蜷缩	浊响	模糊	平坦	硬滑	否
17	青绿	蜷缩	沉闷	稍糊	稍凹	硬滑	否

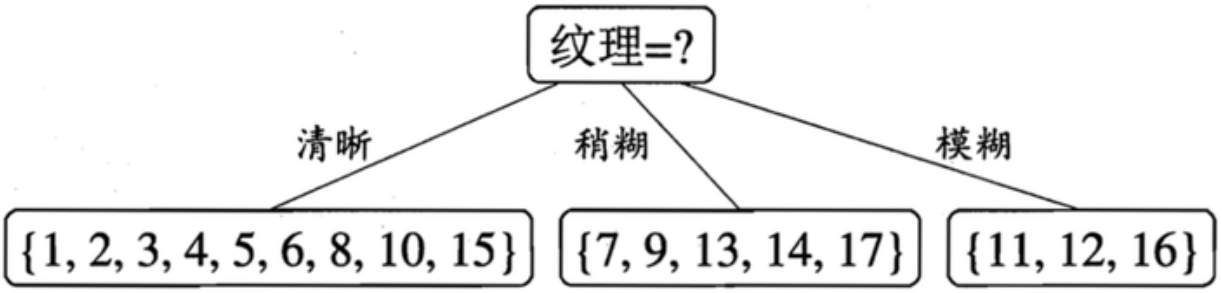
现在选择第一个特征，如果第一个特征选色泽，则根据前面介绍的方法，计算出的信息增益为 0.109，即 $Gain(D, \text{色泽}) = 0.109$。同理，如果选择其他特征，则信息增益如下：

$$\text{Gain}(D, \text{根蒂}) = 0.143; \quad \text{Gain}(D, \text{敲声}) = 0.141;$$

$$\text{Gain}(D, \text{纹理}) = 0.381; \quad \text{Gain}(D, \text{脐部}) = 0.289;$$

$$\text{Gain}(D, \text{触感}) = 0.006.$$

而该算法的逻辑是每次进行特征选择时，选取**最大信息增益**的特征。而根据计算结果，纹理是信息增益最大的特征。因此该决策树的第一层就构建完成了，其结构见下图：



使用纹理这一特征将树的第一层构建完成之后，继续**递归**寻找除了纹理之外的特征的信息增益最大的特征。

分别对纹理分类后的三个子节点计算增益，以纹理清晰这一子节点为例。继续计算各个特征带来的信息增益，得到如下计算结果。

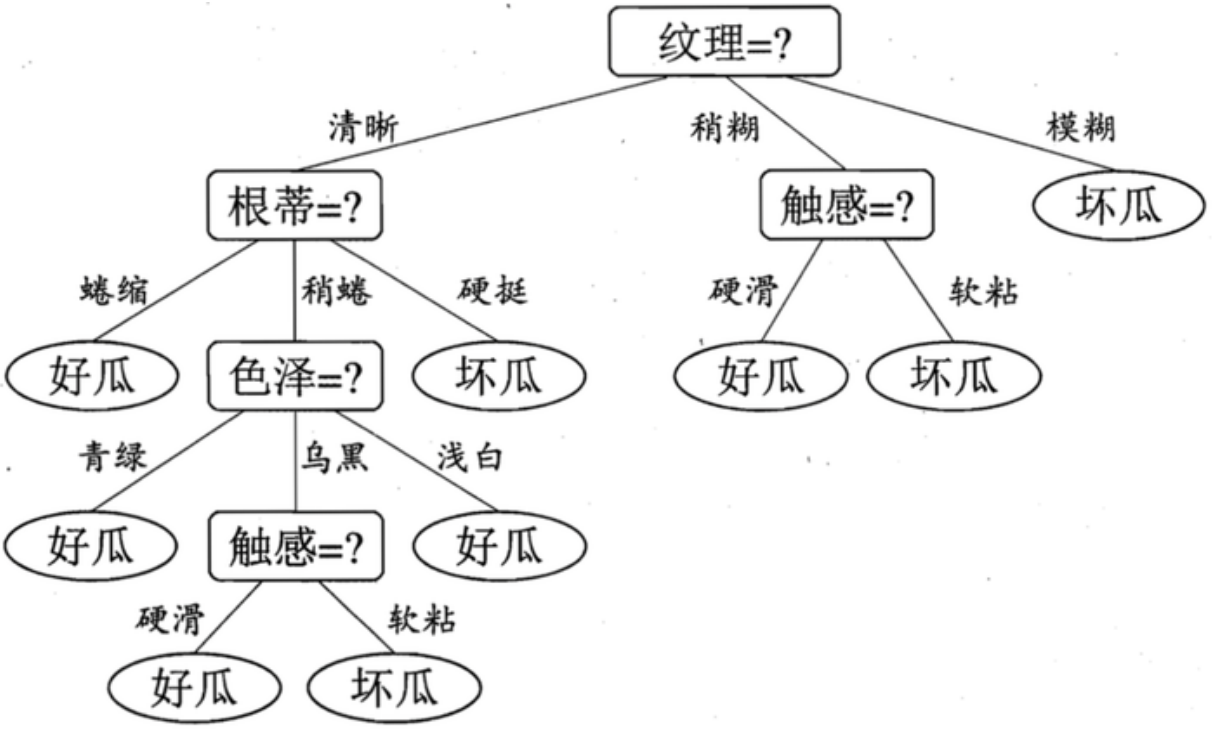
$$\text{Gain}(D^1, \text{色泽}) = 0.043; \quad \text{Gain}(D^1, \text{根蒂}) = 0.458;$$

$$\text{Gain}(D^1, \text{敲声}) = 0.331; \quad \text{Gain}(D^1, \text{脐部}) = 0.458;$$

$$\text{Gain}(D^1, \text{触感}) = 0.458.$$

发现触感、根蒂、脐部的信息增益最大，**三者任选其一**即可。

对这一层递归完成后，得到下图的树结构。



以此类推，**直到叶子节点全部输出**或**无数据集可遍历**即结束。

在这里总结一下，ID3 的决策树算法的流程为：

1. 寻找最优特征，即计算信息增益
2. 将最优特征的特征值作为有向边，判断子数据集。若子数据集均为同一类，则输出叶子节点；反之，若子数据集中仍有特征，重复 1，2 过程，若没有更多特征，则将最多的一类输出为叶子节点。
3. 叶子节点全部输出或无数据集可遍历即结束。

但 $ID3$ 算法的缺点依旧明显，从上述例子可以看出，该方法只适用于计算特征是离散的，比如触感是硬滑还是软粘这种特征，但不能处理比如 $0-1$ 这样的连续特征值。并且用这种方式遍历构建决策树，会有过拟合问题。而且 $ID3$ 倾向于选择大量值的属性（即特征值多的）。为了解决这些问题，引出了 $C4.5$ 算法。

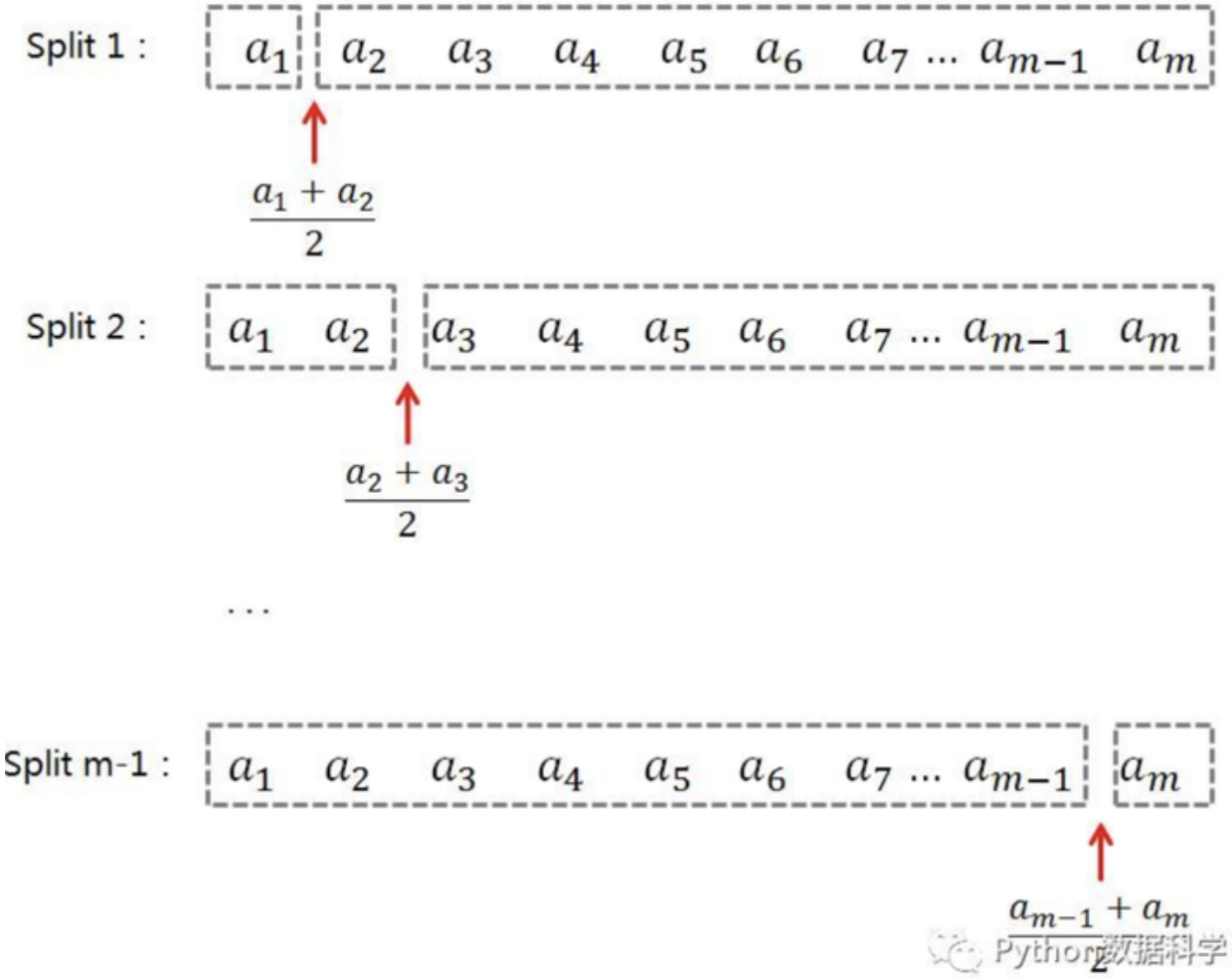
3. C4.5

为了解决特征值数量问题对信息增益计算的影响。该算法使用**信息增益率**替代信息增益作为寻找最优特征的判据。其计算公式就是**信息增益除以子节点的熵。**

$$GainRatio(D,A)=\frac{Gain(D,A)}{-\sum_{i-1}^{n}\frac{|D_i|}{|D|}log_2(\frac{|D_i|}{|D|})}$$

针对 $ID3$ 算法不能处理连续特征的问题，该算法**使用二分法做离散化处理**，先将连续特征从小到大排序，对 N 个连续数据进行 $N-1$ 个二分划分，遍历 $N-1$ 个划分点，分别

计算这些子节点的信息增益，**选择信息增益最大的二分结果作为该连续特征的最佳划分点。**



该方法使用信息增益率解决了 $ID3$ 倾向于特征数量多的特征这一问题，并且实现了应用在连续特征中。并且可以使用剪枝来避免过拟合。

但是，可以看出，使用二分法对连续特征离散化的计算量很大。当连续特征数量比较多的时候，往往需要花费很长的时间去计算连续特征的信息增益率。并且，该算法没有考虑各个特征之间的相关性。

4. CART3

$CART3$ 算法针对分类数，引入了基尼不纯度（又称基尼指数）这一概念，这一概念表示样本集合中在一个随机选中的样本被分错的概率。该指数越小表示该集合的纯度越高，所有样本为同一类时，基尼指数为 0。其中 p_k 表示选中的样本属于第 k 个类别的概率。

$$Gini(p) = \sum_{k=1}^{K} p_k(1-p_k) = 1 - \sum_{k=1}^{K} p_k^2$$

数据集合 D 在特征 A 的条件下分成了 D_1 和 D_2，则集合 D 的基尼指数被定义为：

$$Gini(D,A)=\frac{|D_1|}{|D|}Gini(D_1)+\frac{|D_2|}{|D|}Gini(D_2)$$

分类依据和前两种算法不同，其他都大体相似，在此不过多赘述。本文重点关注 **CART3 回归树**，回归树的构建是用**平方误差最小化准则**进行特征选择，去生成二叉树。

目标准则可以用如下公式来表达，第 j 个特征选择 s 为切分点，使得 $label$ 值和该划分区域的值 c 平方和最小。

$$\min j,s\left[\min c_1\sum_{x_i\in R_1(j,s)}(y_i-c_1)^2+\min c_2\sum_{x_i\in R_2(j,s)}(y_i-c_2)^2\right]$$

第 j 个特征选择 s 为切分点，则划分出了两个集合，其中 x 为特征值，y 为 $label$。

$$R_1(j,s)=\left\{x|x^{(j)}\le s\right\},\quad R_2(j,s)=\left\{x|x^{(j)}>s\right\}$$

被 s 划分出的两个区域，使用这两个区域的均值代表这个两个区域的值，即：

$$c_1=\frac{1}{N_1}\sum_{x_i\in R_1}y_i,\quad c_2=\frac{1}{N_2}\sum_{x_i\in R_2}y_i$$

这是 c 的一般形式，递归至整棵树构建完成。

$$\hat{c}_m=\frac{1}{N_m}\sum_{x_i\in R_m(j,s)}y_i,\quad m=1,2,\cdots$$

该回归树的数学表达式，可理解为将输入空间划分为 M 个区域 $R_1,R_2,\cdots,R_m$ 所生成的决策树。

$$f(x)=\sum_{m=1}^{M}\hat{c}_mI(x\in R)$$

下面给出一个例子，结合例子再回过去看上面的公式和符号，就会比较清晰。假如目标是对下面这一组 x 和 y 做回归。

x	1	2	3	4	5	6	7	8	9	10
y	5.56	5.70	5.91	6.40	6.80	7.05	8.90	8.70	9.00	9.05

该特征为连续特征，前文提到的 $C4.5$ 算法采取的策略是对其进行二分法并逐个计算信息增益率，来寻找最优的切分点。而 $CART3$ 算法则不然，该算法计算不同的切分点的平方误差。以切分点 $s=1.5$ 为例，则该切分点划出了两个区域 $R_1=\{1\}$，$R_2=\{2,3,\cdots,10\}$。计算这两个区域中 y 的平均值，得 $c_1=5.56,c_2=7.50$。

现在需要计算使用 c_1 和 c_2 代表该区域的值是否准确，带入前文的目标准则函数来计算平方误差。

$$m(s)=\min_{j,s}\left[\min_{c_1}\sum_{x_i\in R_1(j,s)}(y_i-c_1)^2+\min_{c_2}\sum_{x_i\in R_2(j,s)}(y_i-c_2)^2\right]=0+15.72=15.72$$

这表明了使用 $s=1.5$ 作为切分点，平方误差为 15.72。同理，算出切分点 s 在 $1.5,2.5,3.5,4.5,5.5,6.5,7.5,8.5,9.5$ 时的 $m(s)$ 如下表。

s	1.5	2.5	3.5	4.5	5.5	6.5	7.5	8.5	9.5
m(s)	15.72	12.07	8.36	5.78	3.91	1.93	8.01	11.73	15.74

可以看出，$s=6.5$ 时，该误差最小。因此在 6.5 处设置切分点是最优选择。

此时，$c_1=6.24$，$c_2=8.91$。则该节点的表达式为：

$$T_1(x)=\begin{cases}6.24, & x<6.5\\ 8.91, & x\geq 6.5\end{cases}\qquad f_1(x)=T_1(x)$$

之后，在子集合中继续递归，直到最终将构建出该回归树模型。

至此，主要的决策树算法已经讲解的差不多了。为了进一步提高树模型的精度，将结合集成学习的思想。集成学习的两大思想分别为 $Boosting$ 和 $Bagging$，结合树模型，则分别对应着 $GBDT$ 和 $RandomForest$ 算法。